

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional San Nicolás
Tecnatura Universitaria en Programación

PROGRAMACIÓN I

Trabajo Práctico Integrador

Gestión de Datos de Países en Python:

filtros, ordenamientos y estadísticas

INTEGRANTES

Nicolás Compagnucci — Comisión 2

Álvaro Leal Gei — Comisión 2

Fecha de entrega: 16 de mayo de 2026

Índice de Contenidos

Índice de Contenidos	2
Marco Teórico	3 - 4
Decisiones Técnicas y Arquitectura	5 - 6
Arquitectura general del sistema	5
Decisión central: trabajo sobre la lista en memoria	5
Funciones del sistema	6
Dificultades y Conclusiones.....	7 - 8
Obstáculos Técnicos	7
Manejo de tildes y caracteres especiales.....	7
Agregar datos nuevos al archivo.....	7
Reflexiones Finales	8
Diagrama de flujo	9

Marco Teórico

Procesamiento de Datos CSV en Python

1. Módulo csv y DictReader

CSV (Comma-Separated Values) es un formato de texto plano para datos tabulares. Python incluye el módulo csv que permite leer y escribir estos archivos sin instalación adicional. La clase csv.DictReader convierte cada fila en un diccionario donde las claves son los encabezados de columna, permitiendo acceder a los datos por nombre (ej. fila['Poblacion']) en lugar de por índice.

Fuente: Python Software Foundation. csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>

2. Manejo de Excepciones: try / except

El bloque try-except permite controlar errores en tiempo de ejecución sin interrumpir el programa. Python ejecuta el bloque try y, si ocurre un error, captura la excepción en except para manejarlo. Se pueden especificar excepciones concretas (ej. ValueError) o usar un except genérico como red de seguridad.

Fuente: Python Software Foundation. Errores y excepciones. <https://docs.python.org/es/3/tutorial/errors.html>

3. Funciones Internas (Anidadas)

Python permite definir funciones dentro de otras funciones. Este patrón encapsula lógica auxiliar que solo tiene sentido dentro de su función contenedora, mejorando la organización del código. En el proyecto se usaron funciones internas como obtener_max(), promedio_poblacion() y registro_continentes().

Fuente: Python Software Foundation. Defining Functions. <https://docs.python.org/3/tutorial/controlflow.html>

4. sorted(), max(), min() y Funciones Lambda

Las funciones sorted(), max() y min() aceptan el parámetro key, que recibe una función para determinar el criterio de comparación. Las funciones lambda son funciones anónimas de una sola expresión, ideales para usarse como key. Ejemplo: sorted(filas, key=lambda x: int(x['Superficie'])) ordena una lista de diccionarios por el valor numérico de la superficie.

Fuente: Python Software Foundation. Sorting Techniques. <https://docs.python.org/es/3/howto/sorting.html>

5. Codificación de Caracteres: Unicode, ASCII y UnicodeEncodeError

ASCII representa 128 caracteres; Unicode supera el millón, siendo un superconjunto de ASCII. Las vocales con tilde (á, é, í, ó, ú) no existen en ASCII, por lo que al intentar codificarlas con .encode('ascii') se lanza un UnicodeEncodeError. El código aprovecha este comportamiento para detectar letras acentuadas mediante ord() y reemplazarlas con su equivalente sin tilde.

Fuente: Python Software Foundation. CÓMO (HOWTO) Unicode. <https://docs.python.org/es/3/howto/unicode.html>

6. Validación de Entrada con Métodos de String

Python incluye métodos nativos para validar cadenas: .strip() elimina espacios al inicio y al final; .isalpha() verifica que todos los caracteres sean letras; .isdigit() verifica que todos sean dígitos; .lower() y .upper() convierten mayúsculas/minúsculas; .replace() sustituye substrings. Se combinaron con bucles while para garantizar que el programa solo avance con entradas válidas.

Fuente: Python Software Foundation. Built-in Types — str. <https://docs.python.org/3/library/stdtypes.html#string-methods>

7. Estructuras de Datos: Listas y Diccionarios

Las listas son colecciones ordenadas y mutables; los diccionarios son estructuras clave-valor. `csv.DictReader` devuelve una lista de diccionarios, donde cada diccionario representa una fila del CSV. Este patrón permite recorrer datos con `for`, filtrarlos con `if`, y acceder a columnas por nombre. La combinación de ambas estructuras es el núcleo del procesamiento de datos del proyecto.

Fuente: Python Software Foundation. Built-in Types. <https://docs.python.org/3/library/stdtypes.html>

Decisiones Técnicas y Arquitectura

Este apartado describe las principales decisiones de diseño que dieron forma a la arquitectura del sistema. El sistema fue estructurado en torno a un único punto de entrada de datos — la función `cargar_csv()` — desde el cual toda la información fluye como parámetro hacia los módulos que la procesan.

Arquitectura general del sistema

El sistema se organiza en tres capas funcionales claramente diferenciadas:

- **Capa de entrada.** La función `cargar_csv()` es la única responsable de leer el archivo `países.csv`. Convierte cada fila en un diccionario y construye la lista `países`, que representa el estado del sistema en memoria durante toda la ejecución.
- **Capa de navegación.** Las funciones `mostrar_menu()`, `menu_ABM()` y `menu_reportes()` actúan como controladores de flujo. Reciben la lista `países` como parámetro y la delegan a cada función operativa según la elección del usuario.
- **Capa operativa.** Las funciones `ABM` y de reportes operan exclusivamente sobre la lista `países` en memoria. Solo las funciones que modifican datos escriben sobre el archivo CSV para persistir los cambios.

Decisión central: trabajo sobre la lista en memoria

La decisión arquitectónica más relevante del proyecto fue separar la carga del archivo CSV del procesamiento de los datos. En la versión inicial, varias funciones abrían el archivo de forma independiente. Esto fue reemplazado por un modelo donde el archivo se lee una única vez al inicio, y la lista resultante se pasa por parámetro a todas las funciones que la necesitan.

- **Consistencia.** Todas las funciones trabajan sobre el mismo estado de datos. Si se agrega o elimina un país, el cambio es inmediatamente visible para cualquier otra función sin necesidad de releer el archivo.
- **Eficiencia.** Se elimina la apertura repetida del archivo en disco. El acceso a datos en memoria es significativamente más rápido que la lectura de archivos.
- **Claridad.** El flujo de datos es explícito: `países` se pasa como argumento visible en cada llamada, lo que facilita la lectura y el mantenimiento del código.

Funciones del sistema

Función	Módulo	Descripción
<code>cargar_csv()</code>	Entrada	Lee el CSV y construye la lista <code>países</code>
<code>mostrar_menu()</code>	Navegación	Menú principal: ABM, Reportes, Salir

Función	Módulo	Descripción
menu_ABM()	Navegación	Submenú de gestión CRUD de países
menu_reportes()	Navegación	Submenú de consultas y estadísticas
agregar_pais()	ABM	Valida y agrega un país a la lista y al CSV
buscar_pais()	ABM	Busca un país por nombre en la lista
mostrar_paises()	ABM	Lista todos los países cargados
actualizar_pais()	ABM	Modifica campos de un país existente
eliminar_pais()	ABM	Elimina un país de la lista y del CSV
filtrar_continente()	Reportes	Filtra países por continente ingresado
filtrar_poblacion()	Reportes	Filtra países por rango de población
filtrar_superficie()	Reportes	Filtra y ordena por rango de superficie
ordenar_nombre()	Reportes	Ordena países alfabéticamente
ordenar_superficie()	Reportes	Ordena por superficie: asc. o desc.
ordenar_poblacion()	Reportes	Ordena por población de menor a mayor
estadisticas()	Reportes	Muestra máx., mín., totales y por continente
limpieza_tildes()	Auxiliar	Normaliza texto eliminando tildes vía ASCII

Dificultades y Conclusiones

Obstáculos Técnicos

A lo largo del desarrollo del proyecto surgieron desafíos técnicos que requirieron investigación adicional y la implementación de soluciones no contempladas en un principio. El obstáculo más significativo estuvo vinculado al procesamiento de caracteres especiales presentes en los datos del archivo CSV.

Por otro lado, uno de los principales desafíos fue el uso de diccionarios en Python y el manejo de archivos de texto, ya que no son herramientas que se utilizan de forma frecuente en la práctica diaria. Fue necesario prestar especial atención al manejo de archivos, en particular a los modos de apertura (`r`, `a`, `w`), para evitar la pérdida o sobreescritura de información.

Manejo de tildes y caracteres especiales

Uno de los problemas más concretos fue la necesidad de comparar cadenas de texto ingresadas por el usuario con los valores registrados en el CSV, específicamente los nombres de continentes. Dado que el archivo contenía valores como "América" o "África" — con tildes — y el usuario podía ingresar la misma cadena sin tildes, las comparaciones directas fallaban de manera silenciosa.

Para resolver este obstáculo se desarrolló la función `limpieza_tildes()`, cuyo funcionamiento se apoya en el sistema de codificación ASCII. La lógica central consiste en intentar codificar cada carácter del texto en ASCII mediante `.encode('ascii')`. Dado que las vocales acentuadas (á, é, í, ó, ú) quedan fuera de ese rango, cualquier intento de codificarlas lanza una excepción `UnicodeEncodeError`. Este comportamiento fue utilizado de forma deliberada como mecanismo de detección.

Una vez identificada la letra acentuada, la función obtiene su valor numérico Unicode mediante `ord()` y lo compara con los valores de las vocales sin tilde para determinar el reemplazo correcto. Esta solución implicó profundizar en conceptos de codificación de caracteres que excedían el contenido trabajado hasta ese momento en el cursado.

Agregar datos nuevos al archivo

Durante el desarrollo se detectó un problema al momento de agregar nuevos datos al archivo. Debido a una validación incompleta, se generaba un renglón vacío, lo que provocó errores en el procesamiento posterior de la información.

La solución consistió en modificar la función para que no permitiera continuar la ejecución si el usuario ingresaba una cantidad insuficiente de datos. A partir de esta corrección, se reforzaron las validaciones, garantizando que todos los campos requeridos sean completados antes de escribir en el archivo.

Reflexiones Finales

El desarrollo del proyecto permitió consolidar aprendizajes que van más allá de la sintaxis del lenguaje, incorporando una perspectiva más amplia sobre la construcción de programas robustos y confiables. Las siguientes reflexiones resumen las principales conclusiones del grupo:

- **Conciencia sobre los escenarios de entrada del usuario.** Un programa no solo debe funcionar correctamente con datos válidos, sino que debe anticipar y manejar de forma controlada todas las variaciones posibles en el ingreso de información. La combinación de bucles while con métodos de validación de strings (.isalpha(), .isdigit(), .strip()) y bloques try-except resultó ser la herramienta central para garantizar que el programa solo avance con datos que cumplan las condiciones requeridas.
- **Consolidar conocimientos teóricos a través de la práctica.** Se destacó la importancia del manejo cuidadoso de archivos, ya que errores en su manipulación pueden afectar la integridad de la información. Asimismo, se fortaleció la lógica de programación y la capacidad de estructurar correctamente los datos.

En conclusión, la experiencia resultó altamente positiva, ya que permitió mejorar habilidades en Python, especialmente en el uso de diccionarios, archivos y validaciones.

Diagrama de flujo

Diagrama de Flujo — Sistema de Países

